



Université de  
Sherbrooke

**Université de Sherbrooke**

**SQL**

*Tables et contraintes simples*

**SQL\_02**

Christina KHNAISSER ([christina.khnaisser@usherbrooke.ca](mailto:christina.khnaisser@usherbrooke.ca))

Luc LAVOIE ([luc.lavoie@usherbrooke.ca](mailto:luc.lavoie@usherbrooke.ca))

*(les auteurs sont cités en ordre alphabétique nominal)*

*Scriptorum/Scriptorum/SQL\_02c-Tables-et-cles, version 1.0.0.a, en date du 2026-01-13*

*— document de travail, ne pas citer —*

# Plan

|                                    |    |
|------------------------------------|----|
| Introduction . . . . .             | 3  |
| 1. Présentation . . . . .          | 4  |
| 2. Création de tables . . . . .    | 6  |
| 3. Retrait de tables . . . . .     | 13 |
| 4. Modification de tables. . . . . | 13 |
| Conclusion . . . . .               | 20 |
| Références. . . . .                | 21 |
| Définitions . . . . .              | 22 |

# Introduction

Le présent document a pour but de présenter une synthèse du langage de définition des tables et des contraintes de clé qu'on retrouve dans SQL.

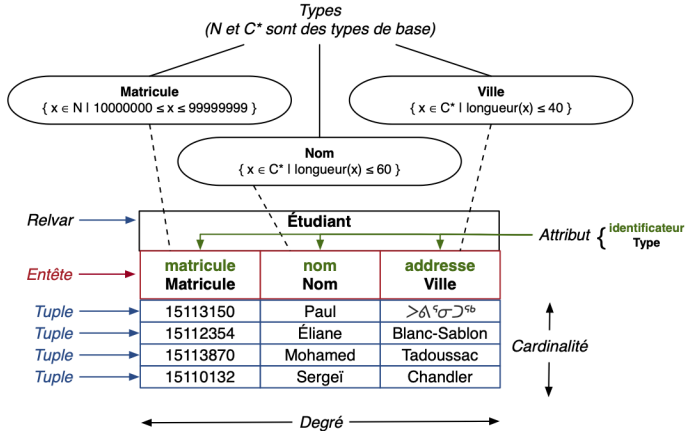
# 1. Présentation

Une relation, en tant qu'objet de la théorie relationnelle, est un ensemble de tuples.

En regard de la théorie des types, il s'agit d'un constructeur (de types non scalaires) applicable à un ensemble de tuples de même type.

Une relation est la représentation d'un prédicat logique. qui correspond un ensemble d'observations de même «nature».

*RELVAR Étudiant {matricule : Matricule, nom : Nom, adresse : Ville}*



En SQL, le constructeur de tables permet de définir une relation.

## 2. Création de tables

### 2.1. Syntaxe (PostgreSQL)

```
CREATE TABLE [ IF NOT EXISTS ] table_name (  
    [{ column_name data_type [ column_constraint [ ... ] ]  
    | table_constraint  
    | LIKE source_table [ like_option ... ] }  
    [, ... ]  
    ] )  
[ INHERITS ( parent_table [, ... ] ) ]
```

where column\_constraint is:

```
{ NOT NULL |  
  NULL |  
  DEFAULT default_expr |  
  GENERATED ALWAYS AS IDENTITY }
```

where table\_constraint is:

```
CONSTRAINT constraint_name  
    { CHECK ( expression ) |
```

```
        UNIQUE [ NULLS [ NOT ] DISTINCT ] ( column_name [, ... ]  
[, column_name WITHOUT OVERLAPS ] ) index_parameters |
```

```
        PRIMARY KEY ( column_name [, ... ] [, column_name WITHOUT  
OVERLAPS ] ) index_parameters |
```

```
        EXCLUDE [ USING index_method ] ( exclude_element WITH  
operator [, ... ] ) index_parameters [ WHERE ( predicate ) ]  
|
```

```
        FOREIGN KEY ( column_name [, ... ] ) REFERENCES reftable  
[ ( refcolumn[, ... ] ) ]
```



```
}  
[ DEFERRABLE | NOT DEFERRABLE ]  
[ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]  
[ ENFORCED | NOT ENFORCED ]
```

## 2.2. Exemple — Bibliovik

La société Bibliovik assure la gestion du réseau des bibliothèques sur le territoire du Nunavik. Elle désire conserver un inventaire des exemplaires de livre qu'elle possède, leur statut, les prêts et les abonnés. Un livre est décrit par un code, un auteur, un titre, un éditeur et une année de parution. Une bibliothèque est décrite par un code et le nom du village où elle se trouve. Chaque exemplaire est décrit par une référence au livre, une référence à la bibliothèque qui le possède, son numéro d'exemplaire, sa date d'acquisition et son statut (disponible, perdu). Finalement, on consigne les prêts (exemplaire, date d'emprunt, date de retour, abonné emprunteur) et les abonnés (numéro d'abonné, nom, prénom, adresse).

| abonne  |              |
|---------|--------------|
| nom     | varchar(60)  |
| prenom  | varchar(240) |
| adresse | varchar(60)  |
| abonne  | char(8)      |

| pret    |                       |
|---------|-----------------------|
| retour  | date                  |
| abonne  | char(8)               |
| statut  | bibliovik.statut_pret |
| livre   | bibliovik.code_livre  |
| biblio  | char(4)               |
| noex    | numeric(2)            |
| emprunt | date                  |

| livre   |                      |
|---------|----------------------|
| auteur  | varchar(60)          |
| titre   | varchar(240)         |
| editeur | varchar(60)          |
| annee   | bibliovik.annee      |
| livre   | bibliovik.code_livre |

| bibliotheque |             |
|--------------|-------------|
| village      | varchar(40) |
| biblio       | char(4)     |

| exemplaire |                        |
|------------|------------------------|
| acquis     | date                   |
| statut     | bibliovik.statut_livre |
| livre      | bibliovik.code_livre   |
| biblio     | char(4)                |
| noex       | numeric(2)             |

← abonne

→ livre, biblio, noex

livre

biblio

*Tableau 1. Dictionnaire des types*

| Nom          | Domaine des valeurs                                                                     | Description                                                  |
|--------------|-----------------------------------------------------------------------------------------|--------------------------------------------------------------|
| Code_livre   | Chaine de caractère qui débute par deux lettres en majuscules suivies de huit chiffres. | Un code unique d'un livre attribué en fonction de son sujet. |
| Annee        | Entier                                                                                  | Un entier qui représente une année après 1600.               |
| Statut_livre | Caractère                                                                               | D = disponible, P = Perdu                                    |

## 3. Retrait de tables

### 3.1. Syntaxe (PostgreSQL)

```
DROP TABLE [ IF EXISTS ] name [, ...] [ CASCADE | RESTRICT ]
```

### 3.2. Exemple — Bibliovik

## 4. Modification de tables

### 4.1. Syntaxe (PostgreSQL)

```
ALTER TABLE [ IF EXISTS ] name
```

```
action [, ... ]
```

where action is:

[ RENAME | ADD | DROP | ALTER | SET ]

## *Renommer*

```
RENAME [ COLUMN ] column_name TO new_column_name
```

```
RENAME CONSTRAINT constraint_name TO new_constraint_name
```

```
RENAME TO new_name
```



## *Modifications d'une colonne*

```
ADD COLUMN column_name data_type [ column_constraint ]
```

```
DROP COLUMN column_name [ RESTRICT | CASCADE ]
```

```
ALTER COLUMN column_name  
  [ TYPE data_type |  
    SET DEFAULT expression |  
    {SET | DROP} SET NOT NULL |
```

## *Modifications d'une contrainte*

```
ADD table_constraint
```

```
DROP CONSTRAINT constraint_name  
  [ DEFERRABLE | NOT DEFERRABLE ]  
  [ INITIALLY DEFERRED | INITIALLY IMMEDIATE ]  
  [ ENFORCED | NOT ENFORCED ]
```

```
ALTER CONSTRAINT constraint_name
```

## 4.2. Exemple — Bibliovik

# Conclusion

# Références

## PG

[fr] <https://docs.postgresql.fr/18/index.html> (2026-01-05)

[en] <https://www.postgresql.org/docs/18/index.html> (2026-01-05)

# Définitions

## SQL (*Structure Query Language*)

Langage de programmation axiomatique fondé sur un modèle inspiré de la théorie relationnelle proposée par E. F. Codd.

[Normes applicables : ISO 9075:2016, ISO 9075:2023]

Produit le 2026-01-20 18:22:29 UTC



**Université de Sherbrooke**